Generación de efectos hápticos en MiYoke_PD_V2.3

Fundamento, trazabilidad, ganancias y límites de salida

Documento técnico del proyecto

4 de octubre de 2026

Resumen

Se describe el código actual del puente SimConnect SC6 y del firmware Pico 2: sensación prevista, datos de Prepar3D, ecuaciones, antecedentes de software y relación entre ganancias y PWM. El sistema produce comandos normalizados. No mide todavía fuerza o torque y no dispone de una calibración que permita afirmar valores en N o N m. Los rangos numéricos son límites de cálculo y no mediciones del esfuerzo de un Cessna R182. Se diferencia una adaptación de TelemFFB de una implementación propia y de un modelo físico verificable.

Índice

1. Qué se siente y qué efectos existen **3**

2. Arquitectura, magnitudes y límites comunes **3**

2.1. Recorrido de los datos 3

2.2. Notación y conversión a PWM 4

2.3. Simbología 4

2.4. Por qué todavía no se puede indicar una fuerza en newtons 5

2.5. Del PWM a la fuerza física 5

3. Variables de Prepar3D obtenidas por SimConnect **5**

4. Centrado dinámico y rigidez de reposo **6**

4.1. Detalle 7

4.2. Fuente bibliográfica del centrado y del reposo 7

5. Turbulencia y ráfagas **8**

5.1. Detalle 8

5.2. Fuente bibliográfica de turbulencia 9

6. Síntesis común de vibraciones **9**

6.1. Explicacion cómo se construye una vibración 9

7. Buffet de pérdida **10**

7.1. Detalle 10

7.2. Fuente bibliográfica del buffet de pérdida 11

8. Buffet del tren retráctil **11**
8.1. Detalle . . . 11
8.2. Fuente bibliográfica del buffet del tren . . . 12

9. Buffet de spoilers **12**
9.1. Detalle . . . 12
9.2. Fuente bibliográfica del buffet de spoilers . . . 12

10. Vibración de rodaje **13**
10.1. Detalle . . . 13
10.2. Fuente bibliográfica del rodaje . . . 13

11. Amortiguación y fricción **14**
11.1. Velocidad utilizada . . . 14
11.1.1. Detalle . . . 14
11.2. Damper . . . 14
11.2.1. Detalle . . . 15
11.2.2. Fuente bibliográfica del damper . . . 15
11.3. Fricción suavizada . . . 15
11.3.1. Detalle . . . 15
11.3.2. Fuente bibliográfica de la fricción . . . 16
11.4. Suma, fundamento y procedencia . . . 16
11.4.1. Explicación de la suma de resistencias . . . 16

12. Ganancias y rangos de salida **17**
12.1. Nombre exacto, pasos y defaults . . . 17
12.2. Ejemplos con defaults y total 60 % . . . 17
12.3. Tabla para cada valor de ganancia a total 60 % . . . 18

13. Trazabilidad y alcance de la fundamentación **20**
13.1. Qué se puede defender en la exposición . . . 20

1. Qué se siente y qué efectos existen

La sensación descrita supone un actuador conectado, con polaridad correcta y capacidad mecánica suficiente. En la etapa declarada del proyecto se observan cálculos y PWM en un LCD; la generación de una señal no prueba que esa sensación haya sido reproducida físicamente.

Efecto	Sensación prevista en el yugo	Ajuste del menú
Centrado dinámico	Resistencia al apartar el mando del centro; aumenta con presión aerodinámica. Permanece al sostener una posición.	Centrado pitch; Centrado roll
Rigidez en reposo	Resistencia residual cuando la presión es pequeña. Es parte del centrado, no una vibración adicional.	Centro reposo
Turbulencia / ráfagas	Empujes variables hacia ambos lados y adelante/atrás asociados a cambios del viento. No es un tono periódico fijo.	Turbul. pitch; Turbul. roll
Buffet de pérdida	Vibración creciente al aproximarse al ángulo de pérdida. Portadora triangular de 13 Hz.	Buffet perdida
Buffet del tren	Vibración con tren retráctil extendido y velocidad suficiente, a 10 Hz. No incluye el golpe de bloqueo del mecanismo.	Buffet tren
Buffet de spoilers	Vibración de 15 Hz al extender spoilers a velocidad suficiente. En el addon R182 analizado su salida es cero.	Buffet spoiler
Rodaje	Vibración de 8 Hz al desplazarse sobre el suelo, predominante en roll. Se anula en reposo y en vuelo.	Vib. suelo
Amortiguación viscosa	Resistencia opuesta a la velocidad del mando; al mover más rápido aumenta hasta su límite.	Damper
Fricción suavizada	Resistencia al movimiento que tiende a una meseta. Se anula en reposo; no simula adherencia estática exacta.	Friccion

Son ocho familias de efectos, contando rigidez residual dentro del centrado. El menú contiene doce ganancias: la total, dos centrados, reposo, dos turbulencias, suelo, tres buffets, damper y fricción. La vista EFECTOS contiene once interruptores, excluyendo la ganancia total. No se implementan actualmente trim físico, flujo de hélice en el centrado, vibración del motor, golpe de aterrizaje separado, movimiento hidráulico del tren ni fuerza en pedales, aunque TelemFFB pueda incluir funciones semejantes.

2. Arquitectura, magnitudes y límites comunes

2.1. Recorrido de los datos

Prepar3D calcula el estado del avión. `SimConnectBridge/main.cpp` solicita veinte variables, calcula presión normalizada, ráfagas y envolventes; envía tramas SC6 de 26 bytes. `SimTelemetry.h` valida rango, CRC y secuencia. `SimEffects.h` sintetiza y suma los efectos; `MotorDriver.h` convierte el comando a PWM. MobiFlight proporciona los ejes al simulador; el firmware consulta esas posiciones con `YOKE_USE_SIMULATION=1`. Los encoders son una alternativa futura, no la fuente actual.

El puente limita los envíos normales a intervalos de al menos 20 ms; el firmware calcula nominalmente cada 2 ms. El CSV del puente se escribe por muestra recibida y no equivale a un registro de paquetes enviados. Un watchdog de 300 ms y la desconexión anulan los efectos. La calibración permite pausa o ausencia de enlace; sus salidas de motor son cero. Ausencia de enlace no prueba que el proceso Prepar3D esté cerrado.

2.2. Notación y conversión a PWM

Sean $G \in [0,1]$ la ganancia total, $g_i \in [0,1]$ una ganancia de efecto, $e_i \in \{0,1\}$ su interruptor y $\widetilde{g}_i = e_i g_i$ la ganancia efectiva. En las fórmulas siguientes se supone activado el efecto; si se desactiva se reemplaza su ganancia por cero. $x_R, x_P \in [-1,1]$ son posiciones normalizadas; v_R, v_P son recorridos centro–extremo por segundo.

$$\mathrm{clip}(z,a,b) = \mathrm{mín}(b, \mathrm{máx}(a,z)), \qquad \mathrm{sat}(z) = \mathrm{clip}(z,0,1), \tag{1}$$

$$\mathcal{R}(z;a,b) = \mathrm{sat}\left(\frac{z-a}{b-a}\right), \qquad L = \frac{220}{255} = 0.862745\ldots \tag{2}$$

La suma por eje es $u_a = \mathrm{clip}(\sum_i u_{a,i}, -L, L)$. La polaridad configurada es $d_a = \pm 1$; actualmente es positiva en ambos ejes. Con `PWM_MAX=255`:

$$PWM_a = \mathrm{clip}\left(\mathrm{sgn}(d_a u_a)\, \lfloor 255|u_a| + 0.5 \rfloor, -220, 220\right). \tag{3}$$

PWM firmado positivo/negativo selecciona la dirección del puente H. Su magnitud es un entero; 220 representa aproximadamente 86.3 % del ciclo útil de escala 255. El driver vuelve a limitar la salida a ese techo. **No es un límite de corriente ni una garantía térmica del motor.**

Para compactar rangos se define

$$\mathcal{P}(z) = \mathrm{mín}\{220, \lfloor 255z + 0.5 \rfloor\}, \qquad z \geq 0. \tag{4}$$

Un rango aislado de comando $[-z, z]$ corresponde al intervalo contenido en $[-\mathcal{P}(z), \mathcal{P}(z)]$. No necesariamente se recorren todos sus enteros en un ensayo. La suma de efectos puede saturar antes que cada efecto aislado; sus máximos no se suman como márgenes independientes.

2.3. Simbología

Una fórmula de este documento combina tres clases de datos: **entrada del simulador**, **ajuste del usuario** y **resultado calculado**. No todas las letras son variables de SimConnect.

R, P y a. R indica roll y P pitch. El subíndice a significa "el eje que se está calculando". No son datos recibidos.

G y g_i. G es Ganancia total: 60 % significa $G = 0.60$. g_i es la ganancia del efecto i, por ejemplo 0.120 para Damper. Ambos vienen del menú, no de Prepar3D.

e_i y $\widetilde{g}_i$. El interruptor vale uno si el efecto está activado y cero si está desactivado. Multiplicarlo por la ganancia permite apagar el efecto sin borrar el ajuste.

u y PWM. u es el comando calculado antes de convertirlo a una salida entera. Su signo selecciona sentido y su magnitud determina PWM. No representa newtons. Por ejemplo, $u = 0.10$ produce $\lfloor 25.5 + 0.5 \rfloor = 26$ PWM.

clip, sat y $\mathcal{R}$. Limitar con clip impide salir de dos extremos; sat limita entre cero y uno. $\mathcal{R}(z;a,b)$ es una rampa: vale cero hasta a, crece linealmente y llega a uno en b. Por ejemplo, $\mathcal{R}(125;100,150) = 0.5$. En esta función a, b son extremos, no ejes.

$|z|$, sgn(z) y mín. $|z|$ toma la magnitud sin signo; sgn devuelve la dirección; mín elige el menor valor. $\lfloor z \rfloor$ es la parte entera hacia abajo. Sumando 0.5 antes se redondea la magnitud al entero más cercano.

$\sum_i$, L, d_a y $\mathcal{P}$. Se suman los comandos de todos los efectos y se limita a $\pm L = \pm 220/255$. d_a corrige el sentido según el cableado/configuración. $\mathcal{P}(z)$ convierte una magnitud normalizada en el máximo PWM entero permitido. Es una abreviatura matemática, no una variable de SimConnect.

El intervalo $[-M, M]$ significa que la salida puede tener ambos signos y no superar la magnitud M. No significa que esté oscilando permanentemente entre esos extremos. Las unidades y la condición de vuelo determinan si se llega a ellos.

2.4. Por qué todavía no se puede indicar una fuerza en newtons

El campo de comando o "F" del LCD es adimensional. La transmisión eléctrica/mecánica requiere, como mínimo,

$$V_m = RI + L_m \frac{dI}{dt} + K_e \omega_m, \quad \tau_m = K_t I, \quad \tau_R \simeq 3\eta_R K_t I_R, \quad F_P \simeq \frac{19\eta_P K_t I_P}{r_{\text{ef}}}. \tag{5}$$

Los reductores declarados son 3:1 y 19:1. Faltan K_t, resistencia, eficiencia, radio o paso de transmisión, pérdidas y mediciones. "Motor 775, 24 V, fuente de 20 A" no determina estas magnitudes. Por ello el rango físico en N y N m es **no determinado** para todos los efectos. La ecuación de motor es un modelo estándar de motor DC, no una fórmula extraída de TelemFFB [6]. Las cotas de comando y PWM sí se determinan en este documento.

2.5. Del PWM a la fuerza física

La tensión aplicada V_m se reparte entre resistencia del bobinado (RI), inductancia ($L_m\, dI/dt$) y tensión generada por el motor al girar ($K_e\omega_m$). I es corriente; R resistencia eléctrica, que en esta ecuación no significa roll; L_m inductancia; ω_m velocidad angular del motor y K_e su constante eléctrica. Ninguna de estas magnitudes se recibe actualmente desde SimConnect.

El torque del motor τ_m depende de la corriente mediante K_t, su constante de torque. Las reducciones 3 y 19 multiplican aproximadamente ese torque y η_R, η_P representan pérdidas de eficiencia. En pitch, dividir por el radio efectivo r_{ef} convierte torque en fuerza lineal F_P. Si la transmisión no es una polea debe usarse la relación mecánica correspondiente. Por eso un PWM conocido no permite deducir por sí solo una fuerza: falta saber cuánta corriente circula y cómo transmite el esfuerzo el mecanismo.

3. Variables de Prepar3D obtenidas por SimConnect

Los nombres y unidades siguientes son los solicitados efectivamente en `main.cpp`, no todos los datos posibles de TelemFFB. El SDK documenta las variables [3].

Variable solicitada	Unidad	Utilización
`DYNAMIC PRESSURE`	psf	Centrado; escala de ráfagas. Conversión a Pa.
`AIRSPEED INDICATED`	knots	Envolventes de pérdida, tren y spoilers.
`AIRCRAFT WIND X`	knots	Componente lateral para ráfagas.
`AIRCRAFT WIND Y`	knots	Componente vertical para ráfagas.

Variable solicitada	Unidad	Utilización
AIRCRAFT WIND Z	knots	Componente longitudinal para ráfagas.
SIM ON GROUND	Bool	Activa rodaje y suprime ráfagas/buffets en tierra.
IS SLEW ACTIVE	Bool	Inhibe cálculo activo en modo desplazamiento.
INCIDENCE ALPHA	degrees	Ángulo de ataque para pérdida.
STALL ALPHA	degrees	Ángulo de pérdida de referencia.
TRAILING EDGE FLAPS LEFT PERCENT	fraction	Promedio de flaps para pérdida.
TRAILING EDGE FLAPS RIGHT PERCENT	fraction	Ídem. Se solicita percent over 100.
DESIGN SPEED VS0	knots	Referencia de velocidad con flaps.
DESIGN SPEED VS1	knots	Referencia de velocidad limpia.
GEAR LEFT POSITION	fraction	Extensión usada por buffet del tren.
IS GEAR RETRACTABLE	Bool	Habilita buffet del tren retráctil.
SPOILERS LEFT POSITION	fraction	Promedio de extensión de spoilers.
SPOILERS RIGHT POSITION	fraction	Ídem.
GROUND VELOCITY	knots	Intensidad del rodaje.
YOKE X POSITION	Position	Posición roll para centrado y derivación de velocidad.
YOKE Y POSITION	Position	Posición pitch para centrado y derivación de velocidad.

El viento está en ejes del avión, no en ejes geográficos. Las fracciones usan 0–1, no 0–100. Además se reciben los eventos de sistema `Pause` y `Sim`; no son fuerzas ni filas adicionales de la definición de veinte variables. No se solicita velocidad mecánica del yugo, corriente ni fuerza. Los datos inválidos de un buffet opcional anulan ese buffet; los fallos de datos esenciales desarman el envío.

4. Centrado dinámico y rigidez de reposo

Sensación: el mando trata de volver a su centro geométrico. La rigidez aumenta con presión; la fuerza persiste si la posición no es cero. No existe desplazamiento del centro por trim en la implementación actual.

$$q = 47.88025898\, q_{\mathrm{psf}}, \qquad q_{\mathrm{ref}} = \tfrac{1}{2}(1.225)(120 \cdot 0.514444444)^2 \simeq 2334.24 \text{ Pa}, \tag{6}$$

$$p = \mathrm{clip}(q/q_{\mathrm{ref}}, 0, 4), \qquad k^* = r + (1-r)p, \tag{7}$$

$$k_n = k_{n-1} + \frac{\Delta t}{0.100 + \Delta t}(k^* - k_{n-1}), \qquad u_{C,a} = -G g_{C,a} k_n x_a. \tag{8}$$

Δt se expresa en segundos y r es Centro reposo. 120 kt es una referencia de escala configurable con `-reference-knots`; no es VNE. Aumentar presión por encima de la referencia sigue aumentando rigidez hasta $p = 4$, límite de representación SC6. Con $r = 0.15$, $0.15 \le k \le 3.55$ en régimen estacionario. Al reiniciar se parte de cero y se filtra.

Ganancias: Centrado pitch (default 0.800), Centrado roll (0.400) y Centro reposo (0.150). Para presión fija, $|u_{C,a}| \le G g_{C,a} k$ y el máximo PWM aislado es $\mathcal{P}(G g_{C,a} k)$. A $p = 0$, el rango es $\pm\mathcal{P}(G g_{C,a} r)$; a $p = 1$, $\pm\mathcal{P}(G g_{C,a})$; en todo el rango SC6, $\pm\mathcal{P}[G g_{C,a}(4 - 3r)]$.

Centro reposo no posee un PWM independiente. Su derivada es $\partial k/\partial r = 1 - p$: subirlo aumenta la rigidez por debajo de la referencia, no la cambia en $p = 1$ y la reduce por encima. Desactivarlo equivale a usar $r = 0$, manteniendo su valor guardado. Esta consecuencia de la interpolación debe explicarse al calibrar.

Procedencia: principio de resorte proporcional a presión y deflexión, compatible con $q = \rho V^2/2$ [4]. La referencia local es `MsfsXpFlightControlsMixIn.py`, métodos `_calculate_dynamic_pressures`

(línea 437), `_calculate_control_coefficients` (563) y `_calculate_joystick_spring_coefficients` (857). TelemFFB utiliza presión, coeficientes y un resorte de dispositivo; también incluye flujo de hélice, offsets y trim que aquí no se trasladaron. La normalización a 120 kt, el residual, el filtro y SC6 son decisiones de este proyecto, no parámetros identificados del R182. Implementación: `main.cpp` y `SimEffects.h::update/command`.

4.1. Detalle

El centrado se calcula en cuatro pasos: leer presión, compararla con una referencia, obtener una rigidez y multiplicarla por cuánto se desplazó el mando.

q_{psf} **y** q**. Entrada SimConnect:** `DYNAMIC PRESSURE`, solicitada en psf. q_{psf} es el número recibido; q es ese mismo dato convertido a pascales multiplicando por 47.88025898. No se lee q de una variable diferente ni se calcula aquí a partir de IAS.

q_{ref}**.** Es una presión de referencia calculada por el puente. 1.225 es la densidad de referencia en kg/m^3; 120 es la velocidad elegida en kt y 0.514444444 convierte kt a m/s. No son tres mediciones del simulador. La ecuación física $q = \rho V^2/2$ explica la referencia: ρ es densidad del aire y V velocidad respecto al aire; el puente utiliza directamente la presión recibida para el cálculo operativo.

p**.** Cociente adimensional q/q_{ref}, limitado entre 0 y 4. $p = 0.5$ significa la mitad de la presión de referencia, no la mitad de la velocidad.

r **y** k^***.** r es la ganancia **Centro reposo**. k^* es la rigidez deseada: parte de r a presión cero y agrega $(1 - r)p$ al aumentar la presión. No son SimVars.

k_n**,** k_{n-1} **y** Δt**.** k_n es la rigidez suavizada actual y k_{n-1} la anterior. Δt es el tiempo entre cálculos del firmware, en segundos. El término $\Delta t/(0.100 + \Delta t)$ decide cuánto se avanza hacia la rigidez deseada; 0.100 s es el ajuste de suavizado. Así no cambia de golpe al llegar una nueva muestra.

x_a**. Entrada SimConnect:** `YOKE X POSITION` para x_R y `YOKE Y POSITION` para x_P. Cero es el centro; magnitud uno es el extremo normalizado del simulador. No se mide todavía el desplazamiento mecánico mediante encoders.

$g_{C,a}$ **y** $u_{C,a}$**.** La ganancia es **Centrado roll** o **Centrado pitch**. El producto $Gg_{C,a}k_n x_a$ calcula el comando. El signo menos hace que se oponga al desplazamiento: con posición positiva, comando negativo.

Ejemplo: con total 60 %, pitch 0.800, reposo 0.150, $p = 0.5$ y $x_P = 0.5$, la rigidez estabilizada es 0.575. El comando es $-0.60 \cdot 0.800 \cdot 0.575 \cdot 0.5 = -0.138$, aproximadamente -35 PWM. Si soltamos hasta posición cero, el centrado pasa a cero aunque la rigidez siga siendo alta. Si se mantiene el mando desplazado, el centrado persiste.

4.2. Fuente bibliográfica del centrado y del reposo

El fundamento físico de la presión dinámica corresponde a NASA Glenn [4]; los nombres y unidades de presión y posición se verifican en el SDK de Prepar3D [3]. El antecedente de software es TelemFFB, `MsfsXpFlightControlsMixIn.py`, métodos `_calculate_dynamic_pressures`, `_calculate_control_coefficients` y `_calculate_joystick_spring_coefficients` [7]. La conversión de unidades, normalización, interpolación con Centro reposo y filtro de 100 ms se verifican en el código de MiYoke [11]. No se atribuye a NASA ni a TelemFFB una calibración del 15 % residual para el R182.

5. Turbulencia y ráfagas

Sensación: perturbaciones no periódicas en pitch y roll asociadas a variaciones del viento. Se suprime en tierra, pausa, Slew o datos no frescos. Para cada componente w_i del viento en kt:

$$a = e^{-\Delta t/1.0}, \qquad \mu_i \leftarrow \mu_i + (1-a)(w_i - \mu_i), \qquad h_i = w_i - \mu_i, \tag{9}$$

$$z_R = 0.70\frac{h_X}{5}p_T\gamma, \qquad z_P = \frac{h_Y + 0.25h_Z}{5}p_T\gamma, \qquad p_T = \mathrm{clip}(p, 0, 1), \tag{10}$$

$$\mathbf{z} \leftarrow \frac{\mathbf{z}}{\max(1, \|\mathbf{z}\|)}, \qquad b = 1 - e^{-\Delta t/0.08}, \qquad \mathbf{s} \leftarrow \mathbf{s} + b(\mathbf{z} - \mathbf{s}). \tag{11}$$

γ corresponde a `-gust-gain` (0–10, lanzador: 1); el 0.25 se puede modificar con `-gust-longitudinal`. Estos ajustes del puente no son ganancias del menú. El firmware aplica:

$$D(s) = \mathrm{sgn}(s)\max\left(\frac{|\,\mathrm{clip}(s,-1,1)| - 0.02}{0.98}, 0\right), \qquad u_{T,a} = Gg_{T,a}D(s_a). \tag{12}$$

Por eje, $|u_{T,a}| \le Gg_{T,a}$ y PWM máximo $\mathcal{P}(Gg_{T,a})$; no implica que ambos ejes alcancen simultáneamente sus máximos por el límite vectorial. Defaults: Turbul. pitch=0.700 y Turbul. roll=0.350.

Procedencia: `TurbulenceModulator.py::update`, filtrado pasaaltos, mezclas 1/0.25/0.7, normalización vectorial y suavizado. La recurrencia local equivale a $h_n = a(h_{n-1} + w_n - w_{n-1})$. Diferencias: constante temporal explícita, escala fija 5 kt, multiplicación por presión, zona muerta, y signo lateral positivo local frente al negativo de la referencia. TelemFFB usa viento relativo y su propia sensibilidad; no se deben intercambiar unidades ni signos sin comprobación. La mezcla es una aproximación háptica, no un cálculo de bisagra del R182. Implementación: `GustModel.h` y `SimTelemetry.h::gustCommand`.

5.1. Detalle

Primero se separa la parte cambiante del viento de su valor habitual; luego se transforma esa variación en empujes de roll y pitch y se suaviza la salida.

w_X, w_Y, w_Z**. Entradas SimConnect:** `AIRCRAFT WIND X`, `AIRCRAFT WIND Y` y `AIRCRAFT WIND Z`, en kt y ejes del avión. i sólo permite escribir la misma operación para cualquiera de las tres componentes.

μ_i **y** h_i**.** μ_i es un promedio que el puente va actualizando. $h_i = w_i - \mu_i$ es lo que se aparta de ese promedio. Un viento constante termina dando h_i cercano a cero; un cambio produce una perturbación transitoria.

$a = e^{-\Delta t/1.0}$**.** a es un coeficiente de memoria, no el eje de salida. e es la base de la función exponencial, no el interruptor e_i. Δt es el tiempo entre muestras del puente y 1.0 s fija qué tan rápido sigue el promedio al viento.

z_R, z_P**.** Son comandos provisionales. Dividir por 5 kt fija la sensibilidad. 0.70 pondera el viento lateral; en pitch se suma la variación vertical y un 25 % de la longitudinal. Son coeficientes del algoritmo, no fuerzas medidas.

p_T **y** γ**.** p_T reutiliza la presión derivada de `DYNAMIC PRESSURE`, pero limitada a uno para este efecto. γ es `-gust-gain` del puente. La ganancia del menú se aplica después; son ajustes distintos.

z y $\|\mathbf{z}\|$**.** El vector reúne los dos comandos provisionales y su norma es $\sqrt{z_R^2 + z_P^2}$. Dividir ambos por el mismo máximo limita la intensidad conjunta sin cambiar su dirección relativa.

b**, s y 0.08.** b es el coeficiente del suavizado de salida; 0.08 s es su constante temporal. **s** guarda la salida filtrada. Estos valores los calcula el puente, no vienen de Prepar3D.

$D(s)$**, 0.02 y 0.98.** El firmware ignora magnitudes hasta 0.02 y reescala lo que queda dividiendo por $1 - 0.02 = 0.98$. Por ejemplo, $s = 0.51$ da $D(s) = 0.5$.

$g_{T,a}$ **y** $u_{T,a}$**.** Son la ganancia **Turbul. roll/pitch** y el comando final del efecto en ese eje. Se multiplica la señal filtrada por esa ganancia y por Ganancia total.

Con $D(s_P) = 0.5$, total 60 % y Turbul. pitch=0.700, el comando es 0.21, aproximadamente 54 PWM. El signo cambia con la perturbación calculada; no se trata de una vibración fija de una frecuencia determinada.

5.2. Fuente bibliográfica de turbulencia

TelemFFB, `TurbulenceModulator.py::update`, es el antecedente de los filtros y de la mezcla de ejes [8]. El SDK documenta las tres variables de viento y la presión [3]. La escala de 5 kt, las constantes temporales explícitas, el signo lateral local y la zona muerta se verifican en `GustModel.h` y `SimTelemetry.h` de MiYoke [11]. Es una adaptación háptica, no una ley experimental de fuerza del Cessna.

6. Síntesis común de vibraciones

Los buffets y el rodaje reciben envolventes $A \in [0, 1]$, filtradas con $\beta = \Delta t/(0.100 + \Delta t)$. La onda triangular usa fase $\phi \in [0, 1000)$:

$$T(\phi) = \begin{cases} 0.004\phi - 1, & \phi < 500, \\ 3 - 0.004\phi, & \phi \geq 500, \end{cases} \qquad \phi_n = (\phi_{n-1} + f\Delta t_{\text{ms}}) \bmod 1000. \tag{13}$$

Su media es cero y su amplitud nominal es uno. El muestreo discreto puede no coincidir exactamente con todos los picos. Cada buffet tiene $u_P = Gg\bar{A}T_f$ y $u_R = \frac{1}{2}Gg\bar{A}T_f$. Por ello sus cotas aisladas son $\pm\mathcal{P}(Gg)$ y $\pm\mathcal{P}(Gg/2)$. La relación 2:1 es un ajuste de este yugo, no una medición aeronáutica.

6.1. Explicacion cómo se construye una vibración

La **envolvente** A indica qué tan intensa debería ser la vibración entre cero y uno. La **portadora** T cambia periódicamente de signo para hacer vibrar el mando. Multiplicarlas produce una vibración pequeña cuando A es pequeño y grande cuando se aproxima a uno.

$\bar{A}$**,** β **y** Δt**.** La barra significa envolvente suavizada. En cada ciclo se calcula $\bar{A}_n = \bar{A}_{n-1} + \beta(A - \bar{A}_{n-1})$. $\beta = \Delta t/(0.100 + \Delta t)$ y Δt está en segundos. Es el mismo tipo de suavizado que el de rigidez.

ϕ**,** f **y** T_f**.** ϕ es la fase interna de la onda, entre 0 y 999; no es un ángulo del avión. f es su frecuencia en Hz. T_f es la triangular generada con esa frecuencia. Aquí f no significa fracción de flap como en la sección de pérdida.

Δt_{ms} **y módulo 1000.** Para avanzar la fase se utiliza el tiempo del firmware en milisegundos. El módulo hace que la fase vuelva al principio al completar un ciclo. La primera rama sube de -1 a $+1$ y la segunda baja de $+1$ a -1.

u_P, u_R**.** Se multiplica la onda por envolvente, ganancia del efecto y ganancia total. Para los tres buffets, roll recibe la mitad del comando pitch. Esa relación es un ajuste del dispositivo.

Ni la fase ni la onda triangular se reciben por SimConnect: se generan en la Pico. SimConnect aporta las condiciones que determinan la envolvente de cada efecto.

7. Buffet de pérdida

Sensación: temblor de 13 Hz que aumenta al acercarse a la pérdida. No es la simulación de un stick shaker instalado en el R182.

$$f = (f_L + f_R)/2, \qquad \alpha_0 = \alpha_s(0.5 + 0.2f), \tag{14}$$

$$V_s(f) = V_{s1} + (V_{s0} - V_{s1})f, \qquad A_s = \text{sat}\left[\mathcal{R}(\alpha; \alpha_0, \alpha_s)\,\text{mín}\left\{(IAS/V_s(f))^2, 1.5\right\}\right]. \tag{15}$$

Se exige avión en vuelo y datos válidos: AoA entre -90 y 90 grados, ángulo de pérdida entre 1 y 45, flaps entre 0 y 1 y velocidades de diseño entre 10 y 400 kt. Datos inválidos producen cero. En los registros del addon $\alpha_s = 16.1°$, por lo que el inicio es 8.05° limpio y 11.27° con flap completo.

Ganancia y rango: Buffet perdida, default 0.240. Usar las cotas comunes de buffet, con envolvente menor que uno fuera de la condición máxima.

Procedencia: `BuffetingEffectMixIn.py::ac_update_buffeting` (línea 93), rama con StallAoA conocido. La interpolación de flaps, umbral AoA y factor de velocidad proceden de ese enfoque. La copia de TelemFFB usa TAS en el factor de flujo; el puente local usa IAS, por lo que no son numéricamente equivalentes a toda altitud. TelemFFB tiene modos Classic y Dynamic y dirección modulada; el firmware usa una sola triangular de 13 Hz. Además satura la envolvente antes de aplicar la ganancia, mientras la referencia incorpora intensidad antes de la saturación final. La frecuencia y umbral no están validados como espectro real R182. Implementación: `BuffetModel.h::calculate` y `SimEffects.h`.

7.1. Detalle

La intensidad crece cuando el ángulo de ataque se acerca a la referencia de pérdida. La posición de flaps modifica desde qué ángulo empieza y qué velocidad se usa como referencia.

f_L, f_R, f**. SimConnect:** `TRAILING EDGE FLAPS LEFT PERCENT` y `TRAILING EDGE FLAPS RIGHT PERCENT`, solicitadas como fracciones 0–1. f es su promedio calculado; no son grados de flap.

α, α_s**. SimConnect:** `INCIDENCE ALPHA` y `STALL ALPHA`, en grados. Son el ángulo actual y el ángulo de pérdida suministrado por el modelo.

α_0**.** Ángulo de inicio del buffet calculado por el puente. Es el 50 % de α_s con flaps limpios y el 70 % con flaps completos. No existe una SimVar que entregue directamente este umbral en la implementación.

V_{s0}, V_{s1}, IAS**. SimConnect:** `DESIGN SPEED VS0`, `DESIGN SPEED VS1` y `AIRSPEED INDICATED`, todas en kt. Son referencias de configuración y velocidad indicada actual; las velocidades de diseño no son una medición instantánea de la pérdida con el peso actual.

$V_s(f)$**.** Referencia interpolada: empieza en V_{s1} y se acerca a V_{s0} al extender flaps. Se calcula localmente.

$\mathcal{R}(\alpha; \alpha_0, \alpha_s)$**.** Vale cero antes del inicio, aumenta linealmente y llega a uno en el ángulo de pérdida. Representa cuánto se ha avanzado dentro de la zona de buffet.

$(IAS/V_s(f))^2$ **y 1.5.** Escalan la envolvente con velocidad. La división elimina las unidades; elevar al cuadrado aproxima una dependencia cuadrática. El factor se limita a 1.5 y después la envolvente total se limita a uno.

A_s**,** $\bar{A}_s$ **y** g_s**.** A_s es la envolvente calculada y $\bar{A}_s$ su versión suavizada. Con $g_s =$ **Buffet perdida**, se usa $u_{s,P} = G g_s \bar{A}_s T_{13}$ y $u_{s,R} = u_{s,P}/2$.

Si el ángulo está a mitad del intervalo entre inicio y pérdida, y $IAS = V_s(f)$, resulta $A_s = 0.5$. Con total 60 % y ganancia 0.240, una vez estabilizado el filtro los picos nominales son unos 18 PWM en pitch y 9 en roll. Esto describe el cálculo, no demuestra que ese umbral coincida con el buffet natural del R182.

7.2. Fuente bibliográfica del buffet de pérdida

TelemFFB, `BuffetingEffectMixIn.py::ac_update_buffeting`, rama con StallAoA conocido, proporciona el antecedente de la envolvente [9]. Los nombres SimConnect de AoA, flaps y velocidades de diseño se verifican en el SDK [3]. La elección de IAS, la saturación previa a la ganancia y la portadora triangular local se verifican en `BuffetModel.h` y `SimEffects.h` [11]. Estos documentos no acreditan que 13 Hz sea una frecuencia medida en el R182.

8. Buffet del tren retráctil

Sensación: vibración de 10 Hz, predominante en pitch, con el tren desplegado durante el vuelo.

$$A_g = g_{\text{ext}}\mathcal{R}(IAS; 100, 150), \qquad u_{g,P} = G g_g \bar{A}_g T_{10}, \quad u_{g,R} = \tfrac{1}{2} u_{g,P}. \tag{16}$$

g_{ext} es la posición del tren izquierdo, no la ganancia. Requiere avión en vuelo, tren retráctil y extensión válida 0–1. Las velocidades son kt; por debajo de 100 no hay envolvente y a partir de 150 llega a su máximo para tren completamente extendido. Esos umbrales no son VLE ni autorización para ensayar fuera de los límites del avión.

Ganancia: Buffet tren, default 0.160; rangos según la síntesis común. **Referencia:** `MotionEffectsMixIn.py::ac_u` (215), rampa de velocidad por extensión, y frecuencia 10 Hz. La copia de TelemFFB usa senoides, umbral adicional de extensión y tratamiento de unidades propio; aquí se usa triangular y extensión izquierda sin ese umbral. El movimiento hidráulico de 150 Hz y el clunk de TelemFFB no se implementaron. Los números 100/150 se interpretan localmente en kt; no se afirma equivalencia numérica con cualquier perfil TelemFFB. Implementación: `BuffetModel.h` y `SimEffects.h`.

8.1. Detalle

Se multiplica cuánto tren está extendido por cuánto avanzó la velocidad dentro del intervalo elegido para el buffet.

g_{ext}**. SimConnect:** `GEAR LEFT POSITION`, entre cero y uno. Es extensión física representada por el simulador, **no una ganancia del menú**, aunque se utilice la letra g.

IAS **y** $\mathcal{R}(IAS; 100, 150)$**.** IAS es `AIRSPEED INDICATED`, en kt. La rampa es cero a 100 kt o menos, 0.5 a 125 kt y uno a 150 kt o más. Los números 100/150 son parámetros locales del efecto.

$A_g, \bar{A}_g$**.** Son la envolvente resultante y su versión suavizada. Por ejemplo, medio tren y media rampa producen $A_g = 0.25$.

g_g, G, T_{10}**.** g_g es **Buffet tren**; G es Ganancia total; T_{10} es la triangular de 10 Hz generada en la Pico. No son datos del simulador.

$u_{g,P}, u_{g,R}$**.** Comandos de este buffet: pitch recibe el producto completo y roll la mitad.

También se consulta `IS GEAR RETRACTABLE` para confirmar tren retráctil y `SIM ON GROUND` para exigir que el avión esté en vuelo. Con tren completo e IAS=125 kt, la envolvente es 0.5. Con total 60 % y Buffet tren=0.160, los picos nominales estabilizados son unos 12 PWM pitch y 6 roll.

8.2. Fuente bibliográfica del buffet del tren

TelemFFB, `MotionEffectsMixIn.py::ac_update_landing_gear`, es el antecedente de la envolvente por velocidad y extensión [10]. El SDK define `GEAR LEFT POSITION`, `IS GEAR RETRACTABLE` y la velocidad indicada [3]. La interpretación local de 100/150 como kt y la onda triangular se verifican en el proyecto [11]; no deben presentarse como límites aeronáuticos certificados.

9. Buffet de spoilers

Sensación: vibración de 15 Hz por la perturbación asociada a spoilers extendidos.

$$A_{sp} = \tfrac{1}{2}(s_L + s_R)\mathcal{R}(IAS; 80, 140), \qquad u_{sp,P} = G g_{sp} \bar{A}_{sp} T_{15}, \quad u_{sp,R} = \tfrac{1}{2} u_{sp,P}. \tag{17}$$

Se exige avión en vuelo y ambas fracciones válidas. **Ganancia:** Buffet spoiler, default 0.160. Rango genérico según las cotas comunes. **Para el R182 instalado, el rango esperado es cero**: el addon declara límite de spoilers cero y los registros aportados contienen valores inválidos que el puente rechaza.

Referencia: `MotionEffectsMixIn.py::ac_update_spoilers` (321): extensión promedio y rampa de velocidad; los umbrales de la referencia se expresan con conversión kt a m/s. TelemFFB combina senoides de 14, 15, 16 y 18 Hz y condiciones adicionales; el proyecto conserva una triangular de 15 Hz. No se implementa el movimiento separado de spoilers. Implementación: `BuffetModel.h` y `SimEffects.h`.

9.1. Detalle

La intensidad depende de cuánto spoiler está extendido y de la velocidad. No depende del movimiento del yugo.

s_L, s_R**. SimConnect:** `SPOILERS LEFT POSITION` y `SPOILERS RIGHT POSITION`, entre cero y uno. Su promedio describe la extensión conjunta usada por el algoritmo.

IAS **y 80/140.** IAS es `AIRSPEED INDICATED`, en kt. Los umbrales 80 y 140 son parámetros del algoritmo: a 110 kt la rampa vale 0.5.

$A_{sp}, \bar{A}_{sp}$**.** Envolvente calculada y filtrada. Si el promedio de extensión es 0.5 y la rampa también 0.5, $A_{sp} = 0.25$.

g_{sp}, G, T_{15}**.** Son **Buffet spoiler**, Ganancia total y triangular de 15 Hz. La Pico aplica $u_{sp,P} = G g_{sp} \bar{A}_{sp} T_{15}$ y entrega la mitad en roll.

`SIM ON GROUND` debe indicar vuelo. Si las posiciones son inválidas, no se inventa una extensión: el resultado se anula. Con total 60 %, ganancia 0.160 y envolvente 0.25, serían aproximadamente 6 PWM pitch y 3 roll en un avión equipado. **En el addon R182 analizado, este ejemplo es sólo demostrativo: no tiene spoilers y la salida esperada es cero.**

9.2. Fuente bibliográfica del buffet de spoilers

TelemFFB, `MotionEffectsMixIn.py::ac_update_spoilers`, fundamenta el antecedente de extensión promedio y escalado por velocidad [10]. El SDK define las posiciones de spoilers y la IAS [3]. La síntesis simplificada a una triangular de 15 Hz y el tratamiento de entradas inválidas pertenecen a `BuffetModel.h` y `SimEffects.h` [11]. No se atribuye al R182 un efecto de una superficie que no equipa este addon.

10. Vibración de rodaje

Sensación: vibración de amplitud creciente con velocidad de suelo, diez veces mayor en roll que en pitch antes del redondeo.

$$A_t = \begin{cases} \text{sat}[(V_{\text{suelo}} - 0.5)/39.5], & \text{en tierra y } V_{\text{suelo}} > 0.5 \text{ kt}, \\ 0, & \text{otro caso}, \end{cases} \tag{18}$$

$$u_{t,R} = \text{clip}(G g_t \bar{A}_t T_8, -40/255, 40/255), \qquad u_{t,P} = u_{t,R}/10. \tag{19}$$

Cuando $A_t = 0$ se borra inmediatamente la oscilación, sin cola de filtro. Se usa velocidad sobre suelo, no IAS: el viento con el avión detenido no debe generar rodaje.

Ganancia: Vib. suelo, default 0.180. Cotas: $|u_R| \leq \text{mín}(G g_t, 40/255)$, $|u_P| \leq \text{mín}(G g_t, 40/255)/10$. PWM máximo: $\text{mín}(40, \lfloor 255 G g_t + 0.5 \rfloor)$ en roll y $\lfloor \text{mín}(255 G g_t, 40)/10 + 0.5 \rfloor$ en pitch. No dividir el PWM ya redondeado.

Procedencia: implementación propia simplificada. TelemFFB `MotionEffectsMixIn.py::ac_update_runway` (116) usa compresión de rueda central y diferencia entre ruedas laterales con pasaaltos. El proyecto no replica esa fórmula: usa velocidad, triangular fija, relación 10:1 y techo 40. No se encontró fundamento R182 para esos coeficientes. No atribuirlos a TelemFFB ni presentarlos como rugosidad medida. Implementación: `GroundModel.h::envelope` y `SimEffects.h`.

10.1. Detalle

El rodaje aumenta su intensidad al aumentar la velocidad sobre el suelo. Usa una onda fija para representar vibración; no está leyendo cada bache de la pista.

V_{suelo}**. SimConnect:** `GROUND VELOCITY`, en kt. Es velocidad respecto al suelo, diferente de `AIRSPEED INDICATED`.

Condición de tierra. `SIM ON GROUND` debe valer uno. Si está en vuelo o la velocidad no supera 0.5 kt, $A_t = 0$.

0.5 y 39.5. 0.5 kt es el umbral para empezar. El divisor 39.5 es $40 - 0.5$: la rampa llega a uno a 40 kt. Son elecciones del programa.

$A_t, \bar{A}_t, T_8$**.** Son intensidad calculada, intensidad filtrada y onda triangular de 8 Hz. La onda es interna a la Pico, no proviene de la pista simulada.

g_t, G **y** $40/255$**.** g_t es **Vib. suelo**; G es Ganancia total. El producto se limita a $40/255$ de comando normalizado para que roll no supere 40 PWM.

$u_{t,R}, u_{t,P}$ **y 10.** El comando pitch es una décima parte del comando roll antes de convertir a enteros. La división por 10 es la relación entre ejes elegida para este efecto, no una reducción mecánica.

Por ejemplo, a 20.25 kt la envolvente vale 0.5. Con total 60 % y Vib. suelo=0.180, una vez estabilizado el filtro, roll alcanza aproximadamente 14 PWM y pitch 1 PWM. Al detenerse se borra inmediatamente el efecto, sin esperar a que desaparezca lentamente el filtro.

10.2. Fuente bibliográfica del rodaje

La fórmula aplicada es propia de MiYoke: `GroundModel.h::envelope` y `SimEffects.h::update/command` [11]. El SDK sustenta el significado de `GROUND VELOCITY` y `SIM ON GROUND` [3]. TelemFFB, `MotionEffectsMixIn.py::ac_update_runway_rumble`, se cita como comparación, pues usa compresión de ruedas y no la fórmula local [10]. No hay una fuente experimental externa que valide los valores 8 Hz, 40 kt o 10:1 para este R182; deben identificarse como parámetros empíricos del prototipo.

11. Amortiguación y fricción

11.1. Velocidad utilizada

No viene directamente de SimConnect: `YokeMotion.h::update` deriva la posición recibida, cuantizada por el protocolo a pasos de 0.0001:

$$v_{\text{bruto}} = \text{clip}\left(\frac{x_n - x_{n-1}}{\Delta t}, -10, 10\right), \tag{20}$$

$$v_n = v_{n-1} + \frac{\Delta t}{0.040 + \Delta t}(v_{\text{bruto}} - v_{n-1}). \tag{21}$$

Ante inversión de signo se reinicia la velocidad filtrada antes de actualizar; después de 40 ms con velocidad bruta de magnitud no mayor que 0.02 se fuerza cero. Un hueco de más de 250 ms reinicializa el estimador; un timestamp repetido no crea velocidad nueva. Son decisiones del proyecto. El eje del simulador puede contener zonas muertas y retardos ajenos al mecanismo; no es una medición directa del encoder.

11.1.1. Detalle

Se calcula qué tanto cambió la posición desde la muestra anterior y se divide por el tiempo transcurrido. Es la misma idea de velocidad igual a distancia dividida por tiempo, pero la posición está normalizada.

x_n, x_{n-1}**.** Posición actual y anterior obtenidas de `YOKE X POSITION` o `YOKE Y POSITION`. El subíndice n identifica la muestra, no una unidad de medida.

Δt**.** Tiempo entre muestras nuevas recibidas por el firmware, en segundos. No se deriva una velocidad nueva al reutilizar el mismo paquete.

v_{bruto}**.** Velocidad calculada antes del filtro. Si la posición pasa de 0 a 0.2 en 0.1 s, resulta 2 recorridos centro–extremo por segundo. No son m/s ni rad/s.

$-10, 10$**.** Límites numéricos de velocidad; evitan que un salto muy grande genere valores arbitrarios.

v_n, v_{n-1} **y 0.040.** Velocidades filtradas actual y anterior. La constante de 0.040 s suaviza los cambios de la derivada. La velocidad que usan los efectos es v_n.

El estimador se reinicia ante pausas largas de datos y detecta detención e inversión de sentido. Estos pasos evitan arrastrar una velocidad ficticia. **No hay una SimVar de velocidad del yugo leída por el puente**: se calcula a partir de las dos posiciones mencionadas.

11.2. Damper

Sensación: oposición al movimiento proporcional a su rapidez, incluso pasando por el centro. Definiendo $m = \text{máx}(|v| - 0.02, 0)$, $c_P = 1$, $c_R = 1/2$:

$$u_{d,a} = -\operatorname{sgn}(v_a) G c_a \,\text{mín}(g_d m_a, 0.4). \tag{22}$$

Esta expresión corresponde al efecto aislado. **Ganancia:** Damper, default 0.120. Para cualquier ganancia y $|v| \le 10$:

$$|u_{d,a}| \le G c_a \,\text{mín}(9.98 g_d, 0.4), \qquad PWM_{\text{máx},a} = \mathcal{P}[G c_a \,\text{mín}(9.98 g_d, 0.4)]. \tag{23}$$

Con $g_d = 0.120$ se alcanza el techo cuando $|v| \ge 0.02 + 0.4/0.120 \simeq 3.353$ recorridos/s. Más ganancia adelanta la saturación; no aumenta indefinidamente la salida máxima.

11.2.1. Detalle

El damper se parece a un amortiguador: ofrece poca resistencia cuando se mueve lento y más cuando se mueve rápido, hasta su techo. La posición por sí sola no lo activa.

v_a y m_a. v_a es la velocidad filtrada calculada a partir de `YOKE X/Y POSITION`. $m_a = \text{máx}(|v_a| - 0.02, 0)$ descuenta una pequeña zona muerta de 0.02 recorridos/s para no responder a movimientos mínimos.

g_d. Es la ganancia **Damper**. Multiplica esa rapidez útil: con igual velocidad, mayor ganancia significa mayor resistencia antes de saturar.

$\text{mín}(g_d m_a, 0.4)$. Limita la contribución de movimiento a 0.4 antes de aplicar la ganancia total y la relación de ejes. Es un límite de software, no una fuerza de 0.4 N.

$G, c_a, -\operatorname{sgn}(v_a)$. G es Ganancia total. $c_P = 1$ y $c_R = 0.5$ hacen que roll tenga la mitad de amplitud para igual velocidad. El signo negativo hace que el comando se oponga al movimiento, independientemente de la posición del mando.

9.98 y $PWM_{\text{máx},a}$. La máxima velocidad permitida es 10; al restar la zona muerta queda $10 - 0.02 = 9.98$. Se inserta en la fórmula para hallar una cota de comando y luego $\mathcal{P}$ la convierte a PWM. No es el PWM que se produce a cualquier velocidad.

Con $v_P = 1$, $m_P = 0.98$, Damper=0.120 y total 60 %, resulta $u_{d,P} = -0.07056$, aproximadamente -18 PWM. Si el movimiento se invierte, cambia el signo; si se detiene, el estimador acaba en cero y desaparece la resistencia.

11.2.2. Fuente bibliográfica del damper

La resistencia viscosa proporcional a velocidad se corresponde con el término mecánico $B\omega$ del modelo DC documentado por IIT Kharagpur [6]; trasladarlo a un mando lineal da $B\dot{x}$. OpenFFBoard, `EffectsCalculator.cpp`, es el antecedente para mantener damper separado del resorte [5]. La zona muerta, normalización, filtro, ganancia y techo exactos son de `YokeMotion.h` y `SimEffects.h::motionAdjustedForce` [11]. Las posiciones de origen son las SimVars de yugo descritas por el SDK [3]. No se copió un coeficiente físico del R182.

11.3. Fricción suavizada

Sensación: resistencia que se aproxima a una meseta al crecer la velocidad; cero al detenerse.

$$u_{f,a} = -\operatorname{sgn}(v_a) G c_a \,\text{mín}\left(g_f \frac{m_a}{0.2 + m_a}, 0.4\right). \tag{24}$$

Ganancia: Friccion, default 0.030. El máximo del factor suavizado es $h = 9.98/10.18 \simeq 0.980354$:

$$|u_{f,a}| \le G c_a \,\text{mín}(h g_f, 0.4), \qquad PWM_{\text{máx},a} = \mathcal{P}[G c_a \,\text{mín}(h g_f, 0.4)]. \tag{25}$$

No es fricción estática exacta: no retiene una carga con $v = 0$. El parámetro 0.2 es un ajuste de regularización en recorridos/s.

11.3.1. Detalle

La fricción suavizada también se opone al movimiento, pero su intensidad se aproxima a un valor casi constante al aumentar la rapidez. Se diferencia del damper porque no crece linealmente durante todo el intervalo previo al límite.

v_a **y** m_a**.** Son los mismos valores calculados a partir de posiciones SimConnect que utiliza el damper. No se recibe una fuerza de fricción desde Prepar3D.

g_f**.** Es la ganancia **Friccion**, conservada aunque se desactive el efecto.

$m_a/(0.2 + m_a)$**.** Es un factor entre cero y casi uno. Vale cero en reposo; con $m_a = 0.2$ vale 0.5; con velocidades mayores se aproxima a uno. El 0.2 tiene las mismas unidades de velocidad normalizada que m_a y fija la transición suave.

$0.4, G, c_a, -\operatorname{sgn}(v_a)$**.** El 0.4 limita la magnitud del término; G aplica Ganancia total; c_a establece la relación entre ejes y el signo se opone al movimiento. Son parámetros u operaciones del programa.

h **y** $PWM_{\text{máx},a}$**.** Aquí $h = 9.98/(0.2 + 9.98)$ es el mayor factor suavizado permitido por el límite de velocidad. No es el residuo de viento h_i de turbulencia. Se usa sólo para calcular el máximo teórico.

Con $v_P = 1$, $m_P = 0.98$, Friccion=0.030 y total 60 %, el comando es aproximadamente -0.01495, unos -4 PWM. Por eso con esas ganancias es normal obtener números pequeños. Al mantener el mando quieto en una posición desplazada, este efecto se anula: no simula la fuerza necesaria para romper una adherencia estática.

11.3.2. Fuente bibliográfica de la fricción

OpenFFBoard, `EffectsCalculator.cpp`, aporta el antecedente de una condición de fricción independiente del resorte [5]. La regularización racional $m/(0.2 + m)$, la zona muerta y el techo 0.4 son la implementación de MiYoke en `SimEffects.h::motionAdjustedForce`, con velocidad de `YokeMotion.h` [11]. No se afirma que esa función racional esté copiada de OpenFFBoard o TelemFFB. El SDK identifica las posiciones utilizadas para derivar velocidad [3]; no entrega una fuerza de fricción.

11.4. Suma, fundamento y procedencia

Con ambos activados el límite de 0.4 se aplica **a su suma**, no por separado:

$$u_{m,a} = -\operatorname{sgn}(v_a) G c_a \operatorname{mín}\left(g_d m_a + g_f \frac{m_a}{0.2 + m_a}, 0.4\right). \tag{26}$$

Así $u_{m,a} v_a \leq 0$: son cargas resistentes en el modelo. La interpretación de amortiguación corresponde a $F_d = -B\dot{x}$; para fricción se regulariza la ley de Coulomb. No se deducen B o F_c físicos de las ganancias adimensionales.

Referencia de implementación: OpenFFBoard, `Firmware/FFBoard/Src/EffectsCalculator.cpp`, casos de spring, damper y friction [5]. Es antecedente para separarlos; no se copió su ley numérica completa. La función racional, límites, filtrado y factores de eje son implementación local. No atribuir estas fórmulas a un archivo TelemFFB. `SimEffects.h::motionAdjustedForce` conserva el nombre histórico, pero actualmente suma cargas independientes y permite oponerse al movimiento con centrado cero. La suma no multiplica ni deforma las portadoras, salvo la saturación final común.

11.4.1. Explicación de la suma de resistencias

Dentro del paréntesis se suman el término de damper $g_d m_a$ y el de fricción $g_f m_a/(0.2 + m_a)$. Después se aplica **una sola vez** el techo 0.4. No se pueden sumar los dos máximos aislados para afirmar que juntos generan 0.8. Finalmente se aplican sentido, ganancia total y factor de eje.

$u_{m,a} v_a \leq 0$ expresa que el comando resistente no apunta en el mismo sentido que el movimiento. Es una propiedad del modelo normalizado, no una medición de potencia eléctrica. En la interpretación

física $F_d = -B\dot{x}$, F_d sería una fuerza, B un coeficiente viscoso y $\dot{x}$ una velocidad física. Esos valores con unidades todavía no están identificados para el mecanismo; la ganancia Damper es adimensional y no debe llamarse directamente B en N s/m.

12. Ganancias y rangos de salida

12.1. Nombre exacto, pasos y defaults

Menú	Default	Paso	Alcance
Ganancia total	90 %	5 %	Escala común G; máximo 100 %.
Centrado pitch	0.800	0.050	Sólo rigidez pitch.
Centrado roll	0.400	0.050	Sólo rigidez roll.
Centro reposo	0.150	0.050	Interpolación de rigidez de ambos ejes.
Turbul. pitch	0.700	0.050	Sólo ráfagas pitch.
Turbul. roll	0.350	0.050	Sólo ráfagas roll.
Vib. suelo	0.180	0.010	Rodaje roll y un décimo en pitch.
Buffet perdida	0.240	0.010	Pitch y mitad en roll.
Buffet tren	0.160	0.010	Pitch y mitad en roll.
Buffet spoiler	0.160	0.010	Pitch y mitad en roll; cero en este addon.
Damper	0.120	0.010	Resistencia viscosa, dos ejes.
Friccion	0.030	0.010	Resistencia suavizada, dos ejes.

Todas las ganancias de efecto están entre 0 y 1. Los defaults son los del código; no prueban qué valores quedaron guardados en la placa. Los ensayos aportados usaron total 60 %, por eso se adopta ese valor para los ejemplos. Activar un efecto con ganancia cero sigue dando cero. El interruptor conserva la ganancia y el estado se guarda con CRC. Los límites de compilación, validez y comunicación siguen teniendo prioridad.

12.2. Ejemplos con defaults y total 60 %

Los siguientes son máximos aislados redondeados, con estímulo máximo, no salidas constantes. Centrado se evalúa a recorrido completo; buffets con envolvente uno; damper/fricción con velocidad 10. Cada rango es simétrico $[-M, M]$.

Efecto/condición	M_R	M_P
Centrado, $p = 0$, $r = 0.15$	9	18
Centrado, $p = 1$	61	122
Centrado, $p = 4$, $r = 0.15$	217	220
Turbulencia, límite de cada eje	54	107
Buffet pérdida	18	37
Buffet tren	12	24
Buffet spoilers, avión equipado	12	24
Buffet spoilers, addon R182	0	0
Rodaje	28	3
Damper	31	61
Fricción	2	5

A 100 % total, el límite global permanece 220. El término conjunto damper/fricción tiene máximo 102 PWM pitch y 51 roll; rodaje mantiene 40/4. Los demás efectos dependen de ganancia/estado y del techo global. La tabla no afirma fuerza física proporcional a estos números.

12.3. Tabla para cada valor de ganancia a total 60 %

La tabla siguiente recorre $g = 0, 0.01, \ldots, 1$. Para controles con paso 0.05, usar las filas correspondientes al valor seleccionado; las fórmulas anteriores cubren también valores migrados de memoria. Cada celda es el extremo positivo M del intervalo $[-M, M]$ después del redondeo.

L: $\mathcal{P}(0.6g)$, válido para turbulencia de un eje, pitch de un buffet y centrado a $p = 1$, $|x| = 1$. B: $\mathcal{P}(0.3g)$, roll de un buffet. S: rodaje roll/pitch. D: damper roll/pitch a $|v| = 10$. F: fricción roll/pitch a $|v| = 10$. Envolventes menores, posiciones parciales y velocidades menores reducen la salida. Centro reposo se evalúa mediante su fórmula, no la columna L.

g	L	B	S (R/P)	D (R/P)	F (R/P)
0.00	0	0	0/0	0/0	0/0
0.01	2	1	2/0	8/15	1/1
0.02	3	2	3/0	15/31	1/3
0.03	5	2	5/0	23/46	2/4
0.04	6	3	6/1	31/61	3/6
0.05	8	4	8/1	31/61	4/7
0.06	9	5	9/1	31/61	4/9
0.07	11	5	11/1	31/61	5/10
0.08	12	6	12/1	31/61	6/12
0.09	14	7	14/1	31/61	7/13
0.10	15	8	15/2	31/61	7/15
0.11	17	8	17/2	31/61	8/16
0.12	18	9	18/2	31/61	9/18
0.13	20	10	20/2	31/61	10/19
0.14	21	11	21/2	31/61	10/21
0.15	23	11	23/2	31/61	11/22
0.16	24	12	24/2	31/61	12/24
0.17	26	13	26/3	31/61	13/25
0.18	28	14	28/3	31/61	13/27
0.19	29	15	29/3	31/61	14/28
0.20	31	15	31/3	31/61	15/30
0.21	32	16	32/3	31/61	16/31
0.22	34	17	34/3	31/61	16/33
0.23	35	18	35/4	31/61	17/34
0.24	37	18	37/4	31/61	18/36
0.25	38	19	38/4	31/61	19/37
0.26	40	20	40/4	31/61	19/39
0.27	41	21	40/4	31/61	20/40
0.28	43	21	40/4	31/61	21/42
0.29	44	22	40/4	31/61	22/43
0.30	46	23	40/4	31/61	22/45
0.31	47	24	40/4	31/61	23/46
0.32	49	24	40/4	31/61	24/48
0.33	50	25	40/4	31/61	25/49
0.34	52	26	40/4	31/61	25/51
0.35	54	27	40/4	31/61	26/52
0.36	55	28	40/4	31/61	27/54
0.37	57	28	40/4	31/61	28/55
0.38	58	29	40/4	31/61	28/57
0.39	60	30	40/4	31/61	29/58
0.40	61	31	40/4	31/61	30/60
0.41	63	31	40/4	31/61	31/61

g	L	B	S (R/P)	D (R/P)	F (R/P)
0.42	64	32	40/4	31/61	31/61
0.43	66	33	40/4	31/61	31/61
0.44	67	34	40/4	31/61	31/61
0.45	69	34	40/4	31/61	31/61
0.46	70	35	40/4	31/61	31/61
0.47	72	36	40/4	31/61	31/61
0.48	73	37	40/4	31/61	31/61
0.49	75	37	40/4	31/61	31/61
0.50	77	38	40/4	31/61	31/61
0.51	78	39	40/4	31/61	31/61
0.52	80	40	40/4	31/61	31/61
0.53	81	41	40/4	31/61	31/61
0.54	83	41	40/4	31/61	31/61
0.55	84	42	40/4	31/61	31/61
0.56	86	43	40/4	31/61	31/61
0.57	87	44	40/4	31/61	31/61
0.58	89	44	40/4	31/61	31/61
0.59	90	45	40/4	31/61	31/61
0.60	92	46	40/4	31/61	31/61
0.61	93	47	40/4	31/61	31/61
0.62	95	47	40/4	31/61	31/61
0.63	96	48	40/4	31/61	31/61
0.64	98	49	40/4	31/61	31/61
0.65	99	50	40/4	31/61	31/61
0.66	101	50	40/4	31/61	31/61
0.67	103	51	40/4	31/61	31/61
0.68	104	52	40/4	31/61	31/61
0.69	106	53	40/4	31/61	31/61
0.70	107	54	40/4	31/61	31/61
0.71	109	54	40/4	31/61	31/61
0.72	110	55	40/4	31/61	31/61
0.73	112	56	40/4	31/61	31/61
0.74	113	57	40/4	31/61	31/61
0.75	115	57	40/4	31/61	31/61
0.76	116	58	40/4	31/61	31/61
0.77	118	59	40/4	31/61	31/61
0.78	119	60	40/4	31/61	31/61
0.79	121	60	40/4	31/61	31/61
0.80	122	61	40/4	31/61	31/61
0.81	124	62	40/4	31/61	31/61
0.82	125	63	40/4	31/61	31/61
0.83	127	63	40/4	31/61	31/61
0.84	129	64	40/4	31/61	31/61
0.85	130	65	40/4	31/61	31/61
0.86	132	66	40/4	31/61	31/61
0.87	133	67	40/4	31/61	31/61
0.88	135	67	40/4	31/61	31/61
0.89	136	68	40/4	31/61	31/61
0.90	138	69	40/4	31/61	31/61
0.91	139	70	40/4	31/61	31/61
0.92	141	70	40/4	31/61	31/61
0.93	142	71	40/4	31/61	31/61
0.94	144	72	40/4	31/61	31/61
0.95	145	73	40/4	31/61	31/61
0.96	147	73	40/4	31/61	31/61
0.97	148	74	40/4	31/61	31/61
0.98	150	75	40/4	31/61	31/61
0.99	151	76	40/4	31/61	31/61

g	L	B	S (R/P)	D (R/P)	F (R/P)
1.00	153	77	40/4	31/61	31/61

Para otra ganancia total, recalcular con G antes de limitar y redondear: no escalar el entero ya saturado. Para un centrado cualquiera usar $\mathcal{P}(Ggk|x|)$. Para variar Centro reposo a q fija, insertar el valor r en $k = r + (1 - r)p$, manteniendo las dos ganancias de centrado. Esto proporciona el rango para cada valor posible sin atribuir un PWM único a una ganancia aislada de su condición de vuelo.

13. Trazabilidad y alcance de la fundamentación

La fuente de TelemFFB es la copia local `VPforce-TelemFFB-refactor`, dentro del proyecto, contrastada con el repositorio del autor [1, 2]. Se leyó el código, no sólo la documentación comercial. No hay metadatos Git que permitan asignar un commit verificado a esa copia. Los números de línea citados corresponden a la copia auditada y pueden cambiar en la rama remota.

Función local	Antecedente verificable	Tipo de relación
Centrado	`MsfsXpFlightControlsMixIn.py`	Principio adaptado; no traslado completo de trim/propwash.
Ráfagas	`TurbulenceModulator.py`	Filtrado y mezcla adaptados; escala, signo y presión locales.
Pérdida	`BuffetingEffectMixIn.py`	Envolvente adaptada; renderizador simplificado.
Tren	`MotionEffectsMixIn.py`	Rampa por extensión; portadora y condiciones locales.
Spoilers	`MotionEffectsMixIn.py`	Envolvente adaptada; una triangular en lugar de varias senoides.
Rodaje	`MotionEffectsMixIn.py`, sólo como comparación	Fórmula actual propia, distinta de compresión de ruedas.
Damper / fricción	OpenFFBoard `EffectsCalculator.cpp`	Separación conceptual; ecuaciones y filtros locales.

Los tres MixIn están, respectivamente, en `telemffb/sim/msfs_xp/` o `telemffb/sim/base/`; TurbulenceModulator está en `telemffb/utils/`. Se entrega junto al TEX un manifiesto `fuentes_sha256.json` con hashes completos de los archivos auditados, para identificar la evidencia aunque cambie el repositorio. El TEX es autónomo y no necesita ese JSON para compilar.

13.1. Qué se puede defender en la exposición

Se puede afirmar que el sistema transforma telemetría en comandos hápticos mediante relaciones físicas y aproximaciones documentadas, con límites y ganancias reproducibles. No se puede afirmar aún que reproduce exactamente las fuerzas de un Cessna R182: faltan coeficientes de bisagra, transmisión real y validación de carga. Citar un algoritmo de TelemFFB acredita un antecedente de software, no una certificación experimental del avión.

Los ensayos CSV aportados permiten reconstruir respuestas esperadas, pero no contienen PWM medido ni ganancias efectivas de la Pico. El LCD de 10 Hz puede ocultar o aliasar portadoras de 8–15 Hz. Para validar salida deben guardarse posición, velocidad, comando, PWM, ganancias y tiempo de la placa; para validar fuerza se necesita además medición de carga/corriente y caracterización del actuador.

Referencias

[1] Walmis / VPforce. *VPforce-TelemFFB*, rama refactor y copia local auditada. `https://github.com/walmis/VPforce-TelemFFB`. Archivos y métodos individualizados en cada sección.

[2] Rutas verificables de los archivos TelemFFB (rama refactor; su contenido remoto puede cambiar):

- MsfsXpFlightControlsMixIn.py: centrado y controles.
- TurbulenceModulator.py: perturbaciones de viento.
- BuffetingEffectMixIn.py: buffet de pérdida.
- MotionEffectsMixIn.py: tren, spoilers y referencia de rodaje.

[3] Lockheed Martin. *Prepar3D v4 SDK: Simulation Variables*. `https://www.prepar3d.com/SDKv4/sdk/references/variables/simulation_variables.html`.

[4] NASA Glenn Research Center. *Dynamic Pressure*. `https://www1.grc.nasa.gov/beginners-guide-to-aeronautics/dynamic-pressure/`.

[5] Ultrawipf y colaboradores. *OpenFFBoard*, `Firmware/FFBoard/Src/EffectsCalculator.cpp`. `https://github.com/Ultrawipf/OpenFFBoard/blob/master/Firmware/FFBoard/Src/EffectsCalculator.cpp`.

[6] IIT Kharagpur, Virtual Labs. *Study of DC Motor, Theory*. `https://vlabs.iitkgp.ac.in/ctrl/Exp3/theory.html`.

[7] Walmis / VPforce. *MsfsXpFlightControlsMixIn.py*, TelemFFB, rama refactor. Métodos de presión dinámica y coeficientes de controles. Consultar archivo del repositorio. Copia local identificada por el manifiesto SHA-256.

[8] Walmis / VPforce. *TurbulenceModulator.py*, método update. Consultar archivo del repositorio.

[9] Walmis / VPforce. *BuffetingEffectMixIn.py*, método ac_update_buffeting. Consultar archivo del repositorio.

[10] Walmis / VPforce. *MotionEffectsMixIn.py*, métodos ac_update_landing_gear, ac_update_spoilers y ac_update_runway_rumble. Consultar archivo del repositorio.

[11] Proyecto MiYoke_PD_V2.3. *Código fuente local auditado*: `SimConnectBridge/main.cpp`, `GustModel.h`, `BuffetModel.h`, `GroundModel.h`, `SimEffects.h`, `SimTelemetry.h`, `YokeMotion.h`, `GainSettings.h`, `MotorDriver.h` y `mis_defines.h`. Identificación de versiones mediante `fuentes_sha256.json`, entregado junto con este documento. Fuente primaria de los parámetros propios, no una validación experimental del avión.